11 - Lambdas y streams

Pregunta número 1

What is the output?

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;
public class StreamTest {
    public static void main(String[] args) {
        List<String> items = Arrays.asList("apple", "banana", "cherry", "date");
        List<String> result = items.stream()
                                   .filter(s -> s.startsWith("a"))
                                   .collect(Collectors.toList());
        System.out.println(result);
    }
}
```

Seleccione una:

- a. [apple, banana]
- b. [banana, cherry]
- c. [apple]
- d. [apple, cherry]

Respuesta: En este ejercicio se está evaluando el uso básico de la API de Streams en Java, específicamente las operaciones intermedias de filtrado y las operaciones terminales de recolección. Hay que desglosar el flujo de ejecución paso a paso para entender lo que está sucediendo. Se crea una lista de ítems que contiene cuatro elementos de tipo cadena: "apple", "banana", "cherry" y "date". El método .stream() convierte la lista original en un flujo de datos para procesarlo secuencialmente de forma declarativa. Se aplica el método filter utilizando la expresión lambda s -> s.startsWith("a"). Este evalúa cada elemento del flujo y deja pasar únicamente aquellos cuya secuencia de caracteres comience con la letra "a" minúscula. "apple" cumple la condición (se mantiene). "banana", "cherry" y "date" no cumplen la condición (se descartan de manera definitiva para las siguientes etapas). El método collect(Collectors.toList()) recolecta los elementos sobrevivientes del flujo y los empaqueta en una nueva estructura de datos de tipo List. En este punto, la lista resultante solo contiene el elemento "apple". La instrucción System.out.println(result); invoca internamente el método toString() de la colección, el cual formatea la salida estándar rodeando los elementos con corchetes.

Pregunta número 2

Dado el siguiente código. ¿Cuál es el resultado?

```
public class Ordenamiento {
    public static void main(String[] args) {
        List<String> names = new ArrayList<>();
        names.add("Kevin");
        names.add("Andrea");
        names.add("Laila");
        names.add("Josias");

        List<String> namesOrder = names.stream()
                                       .sorted()
                                       .collect(Collectors.toList());

        System.out.println(namesOrder);
    }
}
```

Seleccione una:

- a. Compilation error.
- b. Andrea, Josias, Kevin, Laila
- c. Laila, Kevin, Josias, Andrea
- d. Kevin, Andrea, Laila, Josias

Respuesta: La respuesta correcta es b porque el código utiliza la API de Streams para ordenar la lista de nombres alfabéticamente. El método stream() toma los elementos de la lista en su orden original (Kevin, Andrea, Laila, Josías), y la operación sorted(), al no recibir ningún Comparator, los ordena según su orden natural, que en el caso de String es alfabético. Así, “Andrea” queda primero por empezar con A, seguida de “Josias” (J), “Kevin” (K) y “Laila” (L). Finalmente, collect(Collectors.toList()) reúne ese resultado en una lista nueva que se imprime como [Andrea, Josias, Kevin, Laila], mientras que la lista original names no se modifica. El código compila sin problemas, así que se descarta la opción a; la c sería el orden inverso y la d es simplemente el orden en que se agregaron los nombres.

Pregunta número 3

Dado el siguiente código. ¿Cuál es el resultado?

```
public class Ordenamiento {
    public static void main(String[] args) {
        List<Integer> numbersList = new ArrayList<>();
        numbersList.add(3);
        numbersList.add(1);
        numbersList.add(9);
        numbersList.add(8);

        List<Integer> numbersOrder = numbersList.stream()
                                                .sorted(Collections.reverseOrder())
                                                .collect(Collectors.toList());

        System.out.println(numbersOrder);
    }
}
```

Seleccione una:

- (●) a. 9, 8, 3, 1
- ( ) b. 3, 1, 9, 8
- ( ) c. Compilation error
- ( ) d. 1,3,8,9

Respuesta: En este ejercicio se evalúa el ordenamiento de colecciones mediante la API de Streams y el uso de comparadores personalizados en Java. Analizando el flujo lógico de las operaciones, tenemos primero que se crea y llena la lista numbersList con los elementos enteros en el orden de inserción: 3, 1, 9, 8. Al invocar el método .sorted(), se pasa como argumento Collections.reverseOrder(). Mientras que el orden natural de los números enteros es ascendente (de menor a mayor), este comparador invierte la lógica, ordenando los elementos de forma descendente (de mayor a menor). Los elementos en el flujo se reorganizan internamente a 9, 8, 3, 1. El flujo ordenado se empaqueta de nuevo en una instancia de tipo List mediante Collectors.toList(). Aunque la ejecución estricta del método toString() de una lista de Java imprimiría los valores rodeados por corchetes [9, 8, 3, 1].

Pregunta número 4

¿Cuál es el resultado del siguiente código?

```
public class App {
    public static void main(String[] args) {
        String[] fruits = {
            "banana",
            "apple",
            "pears",
            "grapes"
        };
        Arrays.sort(fruits, (a, b) -> a.compareTo(b));
        for (String s : fruits) {
            System.out.print(" " + s);
        }
    }
}
```

Seleccione una:

- a. banana apple pears grapes
- b. apple banana grapes pears
- c. pears grapes apple banana
- d. Lanza una excepción en tiempo de ejecución

Respuesta: En este ejercicio se evalúa la modificación de arreglos in-place utilizando la clase utilitaria Arrays y la implementación de la interfaz Comparator mediante expresiones lambda. Tenemos que desglosar el flujo de ejecución para entender lo que está sucediendo. Primero se define un arreglo unidimensional de tipo String llamado fruits que contiene los elementos "banana", "apple", "pears" y "grapes" en ese orden de inserción exacto. El programa invoca Arrays.sort() pasando el arreglo y una expresión lambda (a, b) -> a.compareTo(b). Esta lambda actúa como un comparador personalizado que simplemente delega la lógica al método compareTo() nativo de la clase String. En Java, el método compareTo de los objetos String implementa por defecto el orden lexicográfico (alfabético) natural. Al aplicar este ordenamiento alfabético, los elementos dentro del arreglo original se reorganizan internamente a: "apple", "banana", "grapes" y "pears". Finalmente, el bucle iterativo for (String s : fruits) recorre el arreglo ya modificado desde el índice 0 hasta el final, imprimiendo cada cadena precedida por un espacio en blanco mediante System.out.print.

Pregunta número 5

Cuál es el resultado al ejecutar el siguiente codigo?

```
import java.util.Arrays;
import java.util.List;
public class Test {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
        long count = numbers.stream()
                .map(n -> n * 2)
                .filter(n -> n > 5)
                .count();
        System.out.println(count);
    }
}
```

Seleccione una:

- a. 0
- b. 2
- c. 3
- d. 4
- e. Compilation fails.

Respuesta: En este ejercicio estamos evaluando la aplicación en cadena de operaciones intermedias y terminales utilizando la API de Streams en Java. Analicemos el flujo de transformación de datos paso a paso. Primero se inicializa una lista numbers que contiene los valores secuenciales 1, 2, 3, 4, 5. Al invocar .stream(), estos elementos ingresan al flujo de procesamiento. El método map(n -> n * 2) aplica una función de transformación a cada uno de los elementos del flujo original. Cada número se multiplica por dos, dando como resultado un nuevo flujo temporal con los valores: 2, 4, 6, 8, 10. El flujo transformado pasa por la etapa de filtrado mediante la expresión lambda n -> n > 5. Este evalúa si cada elemento es estrictamente mayor que cinco. Los valores 2 y 4 son descartados, mientras que 6, 8 y 10 cumplen la condición y continúan en el flujo. Finalmente, se invoca el método count(), el cual contabiliza el número total de elementos que sobrevivieron hasta esta etapa del procesamiento. Al quedar exactamente tres elementos (6, 8 y 10), el método retorna el valor 3 encapsulado en un tipo de dato primitivo long. Al asignar este valor a la variable count y pasarlo por consola mediante System.out.println, el programa imprime el dígito tres.

Pregunta número 6

Dado el siguiente código: ¿Qué imprime en pantalla el código?

```
import java.util.List;
import java.util.function.Predicate;

class Person {
    String name;
    public Person(String name) {
        this.name = name;
    }
    @Override
    public String toString() {
        return name;
    }
}

public class Test {
    static List<Person> filter(List<Person> input, Predicate<Person> predicate) {
        return input.stream().filter(predicate).toList();
    }
    public static void main(String[] args) {
        var a = new Person("John");
        var b = new Person("Jane");
        var c = new Person("Mark");
        var input = List.of(a, b, c);
        var output = filter(input, p -> !p.name.startsWith("J"));
        System.out.println(output);
    }
}
```

Seleccione una:

- a. [John, Jane, Mark]
- b. null
- c. [Jane]
- d. []
- e. [Mark]
- f. [John, Jane]
- g. [John]

Respuesta: En este ejercicio se está evaluando la filtración de colecciones utilizando la API de Streams, expresiones lambda y la interfaz funcional Predicate<T> en Java. Hay que analizar el flujo de ejecución paso a paso. Primero tenemos que se instancian tres objetos Person: a con el nombre "John", b con el nombre "Jane" y c con el nombre "Mark". Se invoca al método filter() pasándole la lista y la expresión lambda p -> !p.name.startsWith("J"). El operador de negación (!) invierte la condición, por lo que el filtro conserva únicamente a aquellas personas cuyo nombre NO empiece con la letra "J". "John" empieza con "J" → !true = false (se descarta). "Jane" empieza con "J" → !true = false (se descarta). "Mark" no empieza con "J" → !false = true (se conserva). El método .toList() empaqueta el único elemento resultante (c) en una lista. Al ejecutar System.out.println(output);, la lista invoca internamente el método toString() sobre sus elementos. Dado que la clase Person sobrescribe @Override public String toString() para devolver la propiedad name, la lista encierra el valor resultante entre corchetes, generando la cadena [Mark].

Pregunta número 7

What will be the output of the above code?

```
import java.util.function.*;
public class Test {
    public static void main(String[] args) {
        Predicate<String> p = (s) -> s.length() > 5;
        Function<String, Integer> f = String::length;
        Consumer<String> c = (s) -> System.out.println(s.toUpperCase());
        Supplier<String> s = () -> "Hello, world!";
        String testString = "functional";
        System.out.println(p.test(testString));
        System.out.println(f.apply(testString));
        c.accept(testString);
        System.out.println(s.get());
    }
}
```

Seleccione una:

- a. false 9 FUNCTIONAL Hello, world!
- b. A runtime exception is thrown.
- c. Compilation fails.
- d. true 9 FUNCTIONAL Hello, world!
- e. true 10 FUNCTIONAL Hello, world!

Respuesta: Este ejercicio está evaluando las 4 interfaces funcionales del paquete java.util.function (Predicate, Function, Consumer y Supplier). Empecemos con el System.out.println(p.test(testString)); p es un Predicate<String> que evalúa si la longitud de la cadena es mayor que 5 (s.length() > 5). testString es "functional", que tiene 10 caracteres. Como 10 > 5 es verdadero, se imprime true. Seguimos con el siguiente System.out.println(f.apply(testString)); f es una Function<String, Integer> que aplica el método String::length para obtener la longitud de la cadena. La longitud de "functional" es 10. Se imprime 10. Continuemos con c.accept(testString); c es un Consumer<String> que toma una cadena y la imprime en mayúsculas (s.toUpperCase()). Al aceptar "functional", se imprime FUNCTIONAL. Finalizamos con System.out.println(s.get()); s es un Supplier<String> que no recibe argumentos y devuelve la cadena "Hello, world!". Se imprime Hello, world!.

Pregunta número 8

¿Cuál será la salida del código anterior?

```
import java.util.function.Predicate;
class Foo {
    int i;
    Foo(int i) {
        this.i = i;
    }
}
public class Test {
    static boolean compare(Foo foo, Predicate<Foo> predicate) {
        return predicate.test(foo);
    }
    public static void main(String[] args) {
        Predicate<Foo> p = f -> f.i > 0; // line 1
        p = p.or(f -> f.i > 0).and(f -> f.i < 0); // line 2
        boolean b = compare(new Foo(1), p); // line 3
        System.out.println(b);
    }
}
```

Seleccione una:

- a. Compilación falla en la línea 1.
- b. Compilación falla en la línea 3.
- c. true
- d. false
- e. Compilación falla en la línea 2.

Respuesta: Este ejercicio evalúa la composición de funciones lógicas utilizando los métodos predeterminados (default methods) or() y and() de la interfaz funcional Predicate<T>. Hay que ir paso a paso para entender qué está pasando en el ejercicio. En la línea 1 se crea un predicado p que verifica si el campo i es mayor que 0 (f -> f.i > 0). En la línea 2 se reasigna p combinando predicados: p.or(f -> f.i > 0): Esto crea un predicado que es verdadero si i > 0 O si i > 0. Lógicamente, esto es equivalente a simplemente i > 0. .and(f -> f.i < 0): Luego, se le aplica un AND (Y lógico) con la condición de que i < 0. El predicado final p evalúa: (i > 0 || i > 0) && (i < 0), lo que simplificado es: i > 0 && i < 0. En la línea 3: Se llama al método compare con un nuevo objeto Foo(1). Esto significa que el valor de i es 1. Llegamos a una parte muy importante, que es la evaluación del predicado: Se evalúa la condición i > 0 && i < 0 para i = 1: tenemos que 1 > 0 es true y, por ende, que 1 < 0 es false. Entonces, realizando la evaluación de true && false, se nos da como resultado false. Finalmente, tenemos que la variable b almacena false y se imprime en pantalla.

Pregunta número 9

What will be the output of the above code?

```
import java.util.stream.Stream;
import java.util.Objects;

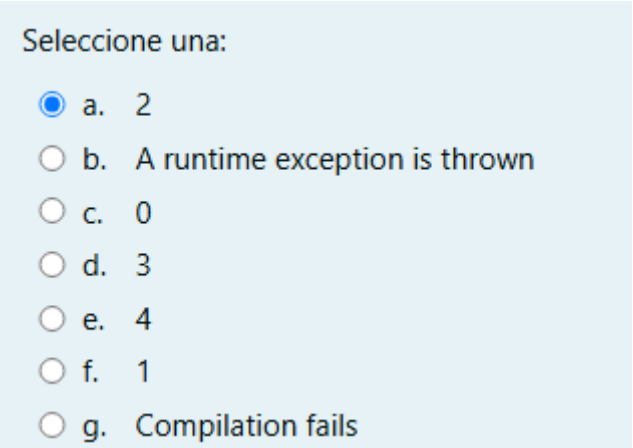
class Person {
    public String name;

    Person(String name) {
        this.name = name;
    }

    @Override
    public int hashCode() {
        return Objects.hash(name);
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (obj == null)
            return false;
        if (getClass() != obj.getClass())
            return false;
        Person other = (Person) obj;
        return Objects.equals(name, other.name);
    }
}

public class Test {
    public static void main(String[] args) {
        Person p1 = new Person("John");
        Person p2 = new Person("Jane");
        Person p3 = new Person("Jane");
        Person p4 = new Person("Mark");
        Stream<Person> stream = Stream.of(p1, p2, p3, p4);
        long count = stream.filter(p -> p.name.startsWith("J")).distinct().count();
        System.out.println(count);
    }
}
```



Respuesta: En este ejercicio se evalúa cómo opera el método .distinct() de la API de Streams sobre objetos de clases personalizadas. Hay que ir desglosando la ejecución paso a paso para entender qué está pasando. Empezamos con la implementación de equals() y hashCode(): La clase Person sobrescribe correctamente ambos métodos basándose en el atributo name. Esto significa que dos objetos Person con el mismo nombre se consideran lógicamente iguales en operaciones basadas en hashing y comparación. Pasamos al filtrado (filter): p.name.startsWith("J") conserva los objetos cuyo nombre inicia con "J": p1 ("John") → pasa el filtro. p2 ("Jane") → pasa el filtro. p3 ("Jane") → pasa el filtro. p4 ("Mark") → descartado. Llegamos a la eliminación de duplicados (distinct): .distinct() utiliza la combinación de hashCode() y equals() para filtrar duplicados. Dado que p2 y p3 tienen el mismo nombre ("Jane"), se consideran idénticos, dejando solo dos objetos únicos en el flujo (p1 y p2). Finalmente, con .count() devuelve el total de elementos sobrevivientes, que es 2.

Pregunta número 10

Given: What will be the output of the above code?

```
interface IFun {
int calculate(int i1, int i2);
}
public class Test {
public static void main(String[] args) {
IFun f = (i1, i2) -> i1 + i2; // line 1
int i1=1, i2 = 2; // line 2
IFun g = (i1, i2) -> i1 * i2; // line 3
System.out.println(g.calculate(i1, i2)); // line 4
}
}
```

Seleccione una:

- a. Compilation fails on line 4.
- b. Compilation fails on line 2.
- c. 2
- d. Compilation fails on line 1.
- e. Compilation fails on line 3.
- f. 3

Respuesta: En este ejercicio se evalúa la regla de ámbito de variables (variable scoping) en expresiones lambda de Java. Primero tenemos en la línea 1 la lambda (i1, i2) -> i1 + i2 declara parámetros llamados i1 e i2. El ámbito de estos parámetros se limita únicamente al cuerpo de esa lambda, por lo que compila sin problemas. En la línea 2 se declaran las variables locales i1 e i2 en el ámbito del método main. Puesto que los parámetros de la línea 1 ya salieron de ámbito, no hay conflicto y la línea 2 compila correctamente. En la línea 3 se produce el error de compilación. A diferencia de las clases anónimas o internas, las expresiones lambda no crean un nuevo ámbito de variables locales independiente para ocultar (shadowing) variables del método contenedor. Al intentar declarar los parámetros de la lambda como (i1, i2) cuando ya existen variables locales con el mismo nombre i1 e i2 en el método main, el compilador arroja un error indicando que la variable 'i1' ya está definida en el ámbito.